

Vetores, Matrizes e Projeções

A geometria escondida atrás de toda regressão, embedding e camada de rede neural

Álgebra linear não é pré-requisito burocrático de machine learning — é a linguagem em que os modelos são escritos. Este material constrói vetores, produto interno, norma, independência linear e posto a partir da geometria, e culmina na projeção ortogonal: a ideia única que explica de uma vez mínimos quadrados, resíduos, R^2 , multicolinearidade e por que similaridade de cosseno funciona em sistemas de busca.

Nível: Introdutório a Intermediário

Pre-requisitos: Python intermediário; nenhum pré-requisito matemático além de álgebra do ensino médio

Carga sugerida: 8 a 10 horas (leitura + 3 notebooks)

Notebooks: 01-algebra-com-numpy · 02-projecao-e-minimos-quadrados · 99-exercicios

Autoria: Material do curso de ML & DL

Versao: 1.0

Sumário

1. Por que este módulo existe	3
2. Vetores: dados como pontos no espaço	4
2.1 Norma: o tamanho de um vetor	4
3. Produto interno: o operador mais importante	5
3.1 Similaridade de cosseno	5
4. Matrizes como transformações	6
4.1 Espaço coluna e posto	6
5. Projeção ortogonal: a ideia que resolve mínimos quadrados	7
5.1 O que a geometria explica de graça	7
6. Como resolver na prática: nunca inverta a matriz	8
7. Ortogonalidade e por que ela facilita tudo	9
8. Erros que custam caro — checklist	10
9. Para ir além	11

1. Por que este módulo existe

Quando você chama `LinearRegression().fit(X, y)`, o que acontece é uma projeção ortogonal. Quando um sistema de busca encontra documentos parecidos, ele calcula ângulos entre vetores. Quando uma rede neural processa um batch, ela multiplica matrizes. Quando um modelo "não converge por multicolinearidade", uma matriz perdeu posto.

Essas quatro frases descrevem a mesma matemática. Aprendê-la uma vez, com geometria, é o que transforma bibliotecas em ferramentas compreensíveis.

ANALOGIA

Um vetor é uma **seta**: tem direção e comprimento. Uma matriz é uma **máquina que transforma setas** — estica, gira, achata. Quase tudo em machine learning é: (1) representar dados como setas, (2) encontrar a máquina que as transforma no que você quer, (3) medir o quanto errou.

2. Vetores: dados como pontos no espaço

Um vetor em $\mathbb{R}^n$ é uma lista ordenada de n números. Em machine learning, quase sempre é uma **observação**: um cliente descrito por (idade, renda, tempo de casa) é um ponto em $\mathbb{R}^3$.

A mudança de perspectiva que importa: uma tabela de 10.000 clientes com 50 colunas é uma nuvem de 10.000 pontos em um espaço de 50 dimensões. Todo modelo de machine learning é uma tentativa de descrever a **forma** dessa nuvem.

2.1 Norma: o tamanho de um vetor

Norma	Como se calcula	Onde aparece
L_2 (euclidiana)	raiz da soma dos quadrados	distância padrão, Ridge, MSE
L_1 (Manhattan)	soma dos valores absolutos	Lasso, MAE, robustez a outliers
L_∞ (máximo)	o maior valor absoluto	pior caso, robustez adversarial

$$\|x\|_2 = \sqrt{\sum_i x_i^2}, \quad \|x\|_1 = \sum_i |x_i|, \quad \|x\|_\infty = \max_i |x_i|$$

NO MERCADO

A escolha entre L_1 e L_2 não é estética. L_2 eleva ao quadrado, então um erro de 10 pesa 100 vezes mais que um erro de 1 — o modelo se dobra para acomodar outliers. L_1 pesa proporcionalmente e ignora extremos. É a mesma razão pela qual a mediana é robusta e a média não: elas minimizam L_1 e L_2 , respectivamente.

3. Produto interno: o operador mais importante

$$\langle x, y \rangle = x^T y = \sum_i x_i y_i = \|x\| \|y\| \cos \theta$$

A segunda igualdade é a que dá sentido geométrico: **o produto interno mede o quanto dois vetores apontam na mesma direção**, escalado pelos comprimentos.

- $\langle x, y \rangle > 0$: ângulo agudo, apontam para o mesmo lado.
- $\langle x, y \rangle = 0$: **ortogonais**, perpendiculares, sem relação linear.
- $\langle x, y \rangle < 0$: apontam para lados opostos.

3.1 Similaridade de cosseno

Normalizando pelos comprimentos, sobra só o ângulo:

$$\cos \theta = \frac{\langle x, y \rangle}{\|x\| \|y\|} \in [-1, 1]$$

NO MERCADO

Essa é a métrica padrão de sistemas de busca semântica, RAG e recomendação baseada em embeddings. A razão de usar cosseno em vez de distância euclidiana: um documento longo e um curto sobre o mesmo assunto têm vetores de comprimentos muito diferentes, mas **mesma direção**. O cosseno ignora o comprimento e captura o assunto.

ARMADILHA COMUM

Correlação de Pearson **é** a similaridade de cosseno entre vetores centrados (com a média subtraída). Isso significa que correlação zero não significa "sem relação" — significa "sem relação **linear**". Duas variáveis com relação perfeitamente quadrática têm correlação zero.

4. Matrizes como transformações

Uma matriz $A \in \mathbb{R}^{m \times n}$ leva vetores de $\mathbb{R}^n$ para $\mathbb{R}^m$. Duas leituras da multiplicação Ax , ambas úteis:

- 1 **Por linhas:** cada entrada de Ax é o produto interno de uma linha de A com x . É a leitura de "cada neurônio calcula uma soma ponderada".
- 2 **Por colunas:** Ax é uma **combinação linear das colunas de A** , com pesos dados por x . É a leitura que explica mínimos quadrados.

A segunda leitura é a que a maioria dos cursos omite e é a mais valiosa.

4.1 Espaço coluna e posto

O **espaço coluna** de A é o conjunto de tudo que se pode alcançar como Ax — todas as combinações lineares das colunas. O **posto** é a dimensão desse espaço: quantas colunas realmente independentes existem.

FORMULARIO

Uma matriz $n \times p$ tem posto no máximo $\min(n, p)$. Se o posto for menor que p , as colunas são **linearmente dependentes**: alguma coluna é combinação das outras e não acrescenta informação nova.

ARMADILHA COMUM

Multicolinearidade é posto deficiente (ou quase). Se você tem as colunas `altura_cm` e `altura_m`, elas são a mesma direção — o posto cai, e $X^T X$ deixa de ser inversível. Na prática o problema é mais sutil: colunas *quase* dependentes deixam $X^T X$ mal condicionada, e os coeficientes ficam instáveis, gigantescos e de sinal imprevisível. O modelo prevê bem e os coeficientes não significam nada. A armadilha do one-hot completo com intercepto é exatamente isso: as dummies somam 1, que é a coluna do intercepto.

5. Projeção ortogonal: a ideia que resolve mínimos quadrados

Aqui está o coração do módulo.

O problema: você quer resolver $X\beta = y$, mas y não está no espaço coluna de X — não existe combinação das features que reproduza exatamente o alvo. O sistema é **inconsistente** (mais equações que incógnitas: $n > p$).

A solução: se não dá para alcançar y , alcance o **ponto do espaço coluna mais próximo de y** . Esse ponto é a projeção ortogonal $\hat{y}$.

ANALOGIA

Você está em um campo aberto (y) e precisa chegar a uma estrada reta (o espaço coluna de X). O ponto mais próximo é o pé da perpendicular. E "perpendicular" é literal: o vetor que liga você à estrada forma 90° com ela.

A condição de ortogonalidade é o que produz a fórmula. O resíduo $y - X\beta$ deve ser ortogonal a **todas** as colunas de X :

$$X^T (y - X\beta) = 0 \implies X^T X\beta = X^T y \implies \hat{\beta} = (X^T X)^{-1} X^T y$$

FORMULARIO

Equações normais: $\hat{\beta} = (X^T X)^{-1} X^T y$.

Matriz de projeção (chapéu): $H = X(X^T X)^{-1} X^T$, com $\hat{y} = Hy$.

Propriedades: H é simétrica, **idempotente** ($H^2 = H$ — projetar duas vezes é o mesmo que projetar uma), e $\text{tr}(H) = p$, o número de parâmetros.

5.1 O que a geometria explica de graça

- **Resíduos ortogonais às features.** Por construção, $X^T e = 0$. Se você

plotar resíduo contra uma feature e vir estrutura, a relação não é linear — porque a parte linear já foi removida por construção.

- **R^2 é um cosseno ao quadrado.** É o cosseno do ângulo entre y centrado e $\hat{y}$ centrado, elevado ao quadrado. $R^2 = 1$ significa ângulo zero.

- **Teorema de Pitágoras:** $\|y - \bar{y}\|^2 = \|\hat{y} - \bar{y}\|^2 + \|e\|^2$

— a decomposição da soma de quadrados total em explicada e residual é literalmente Pitágoras no espaço das observações.

- **Graus de liberdade** são dimensões: n observações, p dimensões consumidas pelo ajuste, $n - p$ restantes para estimar a variância do erro.

6. Como resolver na prática: nunca inverta a matriz

$(X^T X)^{-1}$ é uma expressão matemática, não um algoritmo. Calcular a inversa explicitamente é numericamente instável e mais lento.

Método	Custo	Estabilidade	Uso
Inversa explícita	$O(p^3)$	ruim	nunca
Equações normais + Cholesky	$O(np^2 + p^3/3)$	média	p pequeno, bem condicionado
Decomposição QR	$O(np^2)$	boa	padrão de mercado
SVD	$O(np^2)$	ótima	posto deficiente, <code>lstsq</code>

ARMADILHA COMUM

Formar $X^T X$ **eleva ao quadrado o número de condição** da matriz. Se X tem condição 10^6 — comum quando as features têm escalas muito diferentes —, $X^T X$ tem 10^{12} , e você perde toda a precisão de um float64. Use `np.linalg.lstsq` (que usa SVD) ou `scipy.linalg.lstsq`, nunca `inv(X.T @ X) @ X.T @ y`.

7. Ortogonalidade e por que ela facilita tudo

Quando as colunas de X são ortogonais entre si, $X^T X$ vira diagonal e:

- os coeficientes são **independentes** — remover uma feature não muda as outras;
- a projeção se decompõe em contribuições separadas por feature;
- não há multicolinearidade por construção.

É por isso que PCA (que gera componentes ortogonais) resolve multicolinearidade, e por que desenhos experimentais fatoriais buscam ortogonalidade entre fatores.

NO MERCADO

O processo de **Gram-Schmidt** — ortogonalizar um conjunto de vetores um a um, removendo de cada um a componente já explicada pelos anteriores — é a base da decomposição QR e a intuição por trás de "controlar por uma variável" em regressão múltipla. O coeficiente de x_2 numa regressão com x_1 é o coeficiente da parte de x_2 **ortogonal a** x_1 .

8. Erros que custam caro — checklist

- Inverter $X^T X$ explicitamente em vez de usar QR/SVD.
- One-hot completo junto com intercepto (dependência linear exata).
- Interpretar coeficientes de features colineares como efeitos independentes.
- Usar distância euclidiana em embeddings onde o comprimento não é informativo (use cosseno).
- Esquecer de padronizar features antes de métodos baseados em distância — uma feature em reais domina uma em proporções.
- Confundir correlação zero com independência.
- Ignorar o número de condição antes de interpretar coeficientes.

9. Para ir além

- Strang, *Introduction to Linear Algebra* e a série de aulas do MIT 18.06 — a melhor introdução geométrica que existe.
- 3Blue1Brown, *Essence of Linear Algebra* — a intuição visual em vídeo.
- Trefethen & Bau, *Numerical Linear Algebra* — por que QR e SVD, e não a inversa.
- Boyd & Vandenberghe, *Introduction to Applied Linear Algebra* — orientado a aplicações, com foco em mínimos quadrados.